

COMPARATIVE DATA SYSTEMS STUDY

# OpenAlex LLM Analyzer

A Reproducible Multi-Paradigm Evaluation of **PostgreSQL (Relational)**, **Neo4j (Graph)**, and a **Relational Data Warehouse (Star Schema)** over Scholarly LLM Networks (2020–2025).

POSTGRESQL 3NF

NEO4J LPG

STAR SCHEMA DFM

# Project Goals & Scope

---

## Scientific Objectives

- ✔ **Unified Canonical Corpus:** Reconciles a single frozen OpenAlex corpus to guarantee 100% semantic parity across engines.
- ✔ **Empirical Benchmark:** Directly compares schema expressiveness (SQL vs. Cypher) and warm-cache query timing.
- ✔ **End-to-End Reproducibility:** Completely automated execution from API extraction to database containerization.

## Why LLM Research (2020–2025)?

- ⚡ **High Growth Velocity:** ArXiv preprints lead peer-reviewed venues, creating unique publication dynamics.
- 🔗 **Interconnected Topologies:** Dense networks between corporate AI labs (OpenAI, DeepMind) and universities.
- 📈 **Multi-Dimensional Queries:** Requires path traversals (citation chains) and aggregations (yearly impact).

# End-to-End Pipeline Architecture

---

Deterministic data ingestion, transformation, and multi-model deployment flow:

## 1. Extraction

OpenAlex API cursor  
pagination with 4 search  
seed terms.

## 2. Raw Storage

Immutable JSON  
snapshot with SHA-256  
checksums.

## 3. ETL Engine

10 reconciled CSV  
contracts &  
closed-network  
boundary pruning.

## 4. Multi-Load

Parallel ingestion into  
PostgreSQL 3NF, DW,  
and Neo4j LPG.

# Extraction

A deterministic and restartable OpenAlex acquisition pipeline for the 2020–2025 LLM research corpus.

## FROZEN CORPUS SCOPE

**Define the population before collecting data:**

Every extraction run uses the same temporal window, publication types, seed queries, and relevance rule.

```
START_YEAR, END_YEAR = 2020, 2025
ALLOWED_TYPES =
"article|review|preprint"
SEEDS = [
    "large language model", "generative
AI", "foundation model", "transformer
language model",
]
```

**Fixed scope** → comparable and reproducible datasets

## CURSOR-BASED COLLECTION

**Retrieve all pages safely:**

For every year and every seed query, the collector traverses OpenAlex result pages using a cursor.

```
params = {
    "search": seed,
    "filter": (
        f"from_publication_date:{year}-01-01,"
        f"to_publication_date:{year}-12-31,"
        f"type:{ALLOWED_TYPES}"
    ),
    "sort": "cited_by_count:desc",
    "per_page": "100",
    "cursor": cursor,
}
```

**Cursor pagination** prevents incomplete first-page-only extraction

## AUDITABLE RAW SNAPSHOT

**Preserve the API output before transformation:**

Preserve the API output before transformation

```
filename = (
    f"year_{year}__seed_{seed_number:02d}"
    f"__page_{page_number:04d}.json"
)

output_path.write_text(
    json.dumps(response_data,
ensure_ascii=False, indent=2),
    encoding="utf-8",
)
```

**Raw data and metadata** are separated from credentials

**POST-EXTRACTION PROFILING:** Deduplicate works → validate relevance → rank by citations within each year → retain at most 800 works/year  
Internal citations are kept only when both the citing and cited works belong to the retained corpus.

# ETL ENGINE

Transforms the selected OpenAlex raw snapshot into validated, canonical CSV contracts.

## INPUT

### Frozen raw snapshot:

Reads only work IDs approved in scope\_manifest.json.

```
manifest = json.loads(
    manifest_path.read_text(encoding="utf-8")
)
selected_ids =
selected_ids_from_manifest(manifest)

if missing_selected:
    raise ValueError("Refusing partial ETL")
```

Same selected corpus for every database

## TRANSFORM

### Normalize & flatten:

Converts nested JSON into clean works, entities, and relationships.

```
"work_id": openalex_id(work.get("id")),
"title": clean_text(work.get("title")),
"publication_date": valid_date(
    work.get("publication_date")
),
```

Consistent IDs, text, dates and null values

## VALIDATE & EXPORT

### Canonical CSV contracts:

Validates relationships and exports 10 reusable CSV contracts.

```
if cited_work_id not in selected_ids:
    continue

if cited_work_id == work_id:
    continue

citations.add((work_id, cited_work_id))
```

No external edges · no self-loops · no duplicates

# Multi-Load

The same 10 canonical CSV contracts are loaded into three specialized data models.

## RELATIONAL LOAD

### PostgreSQL 3NF:

Streams each canonical CSV into normalized tables through bulk COPY.

```
statement = sql.SQL(
    "COPY {} FROM STDIN "
    "WITH (FORMAT CSV, HEADER TRUE, "
    "NULL '')"
).format(sql.Identifier(table))

with cur.copy(statement) as copy:
    copy.write(chunk)
```

Fast bulk loading with SQL constraints

## GRAPH LOAD

### Neo4j Property Graph:

Loads entities as nodes and relationships as typed graph edges in batches.

```
UNWIND $rows AS row
MATCH (a:Author {id: row.author_id})
MATCH (w:Work {id: row.work_id})
MERGE (a)-[r:AUTHORED]->(w)
SET r.author_order = row.author_order
```

Typed relationships with idempotent MERGE

## ANALYTICAL LOAD

### Data Warehouse:

Maps canonical IDs to dimensions, facts, and bridge tables for OLAP analysis.

```
source_keys = dimension_map(
    cur, "dim_source", "source_id"
)

cur.execute(
    "INSERT INTO fact_work "
    "(work_id, date_key, source_key, "
    "work_type_key, ...)"
)
```

Surrogate keys for fast analytical joins

## SHARED CANONICAL INPUT

**works · authors · institutions · sources · topics · authorships · affiliations · work\_topics · work\_sources · citations**

One reconciled corpus guarantees semantic parity across relational, graph, and analytical workloads.

Same entities · same relationships ·  
different physical models

# Project Organization

---



# 10 Reconciled Canonical Contracts

| Contract File    | Role / Type   | Primary / Composite Key         | Key Attributes & Descriptors                                          |
|------------------|---------------|---------------------------------|-----------------------------------------------------------------------|
| works.csv        | Core Entity   | work_id                         | Title, publication_date, publication_year, work_type, cited_by_count  |
| authors.csv      | Entity        | author_id                       | Author display_name                                                   |
| institutions.csv | Entity        | institution_id                  | Display_name, country_code, institution_type                          |
| sources.csv      | Entity        | source_id                       | Venue title, publisher, ISSN, source_type                             |
| topics.csv       | Entity        | topic_id                        | Display_name + 4-tier taxonomy (subfield, field, domain)              |
| authorships.csv  | Relation      | (work_id, author_id)            | Author position (first, middle, last), author order, is_corresponding |
| affiliations.csv | Ternary Event | (work_id, author_id, inst_id)   | Work-contextual institutional affiliation                             |
| work_topics.csv  | Relation      | (work_id, topic_id)             | Relevance score (0.0 to 1.0), is_primary_topic flag                   |
| work_sources.csv | Relation      | (work_id, source_id)            | Host publication venue assignment (location_role)                     |
| citations.csv    | Relation      | (citing_work_id, cited_work_id) | Directed internal citation edge between retained works                |

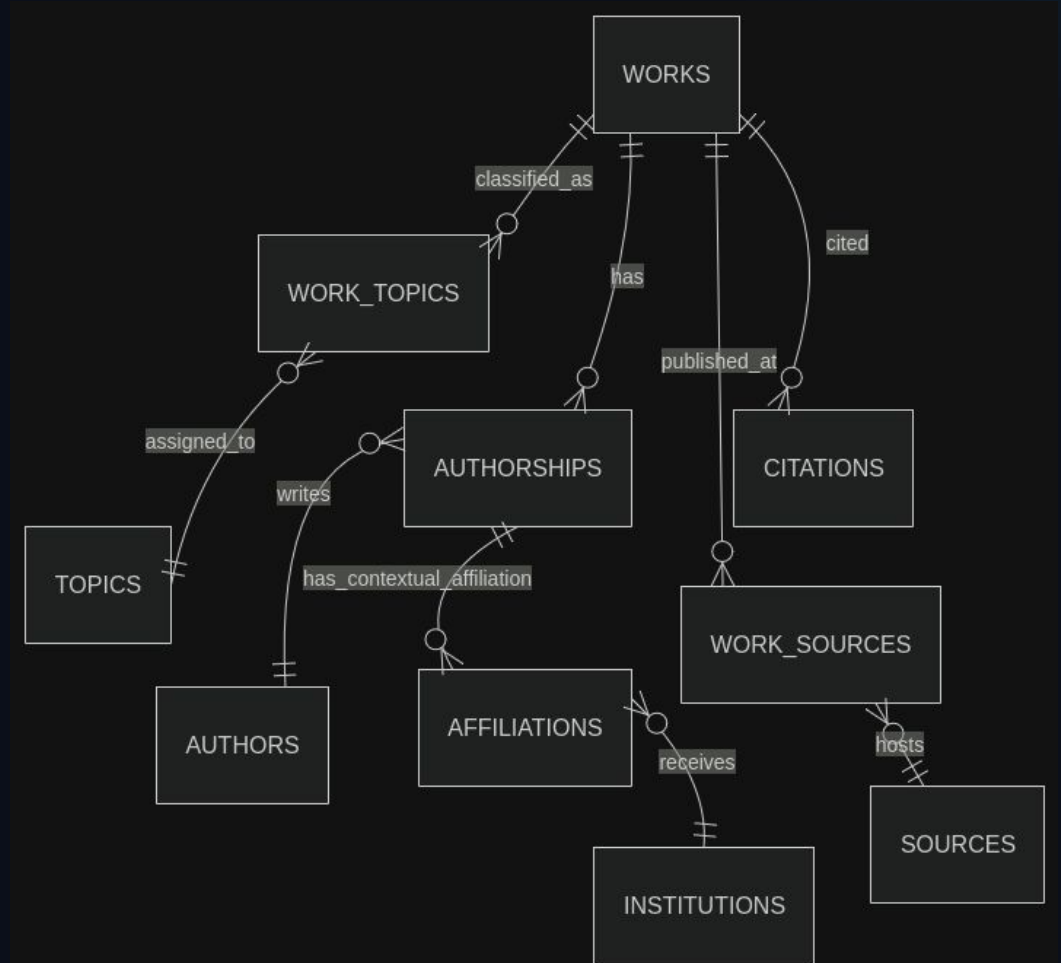
# PostgreSQL Relational Model

## 3NF Schema & Foreign Keys

- 🔒 **Strict ACID Constraints:** Enforces declarative foreign key checks and cascading integrity across all tables.
- ✓ **Domain Validations:** Explicit CHECK constraints for non-negative citations and valid work types.
- ⚡ **Bulk COPY Loading:** High-speed streaming binary copy protocol loads full dataset in <1 second.

## Engineered Indexing Strategy

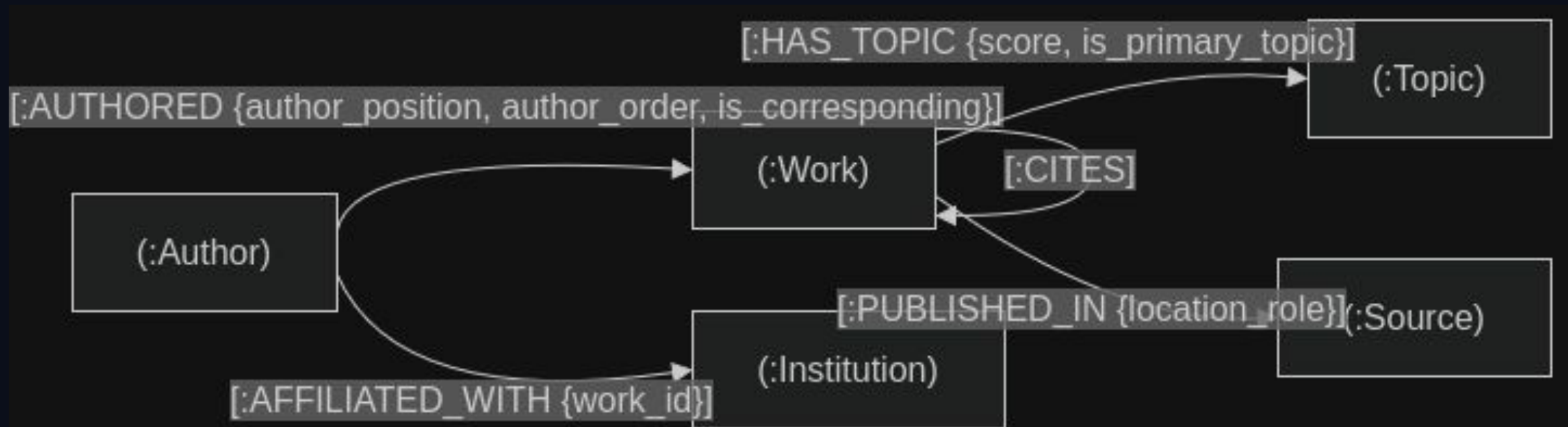
```
CREATE INDEX idx_works_publication_year ON works (publication_year);
CREATE INDEX idx_works_type_year ON works (work_type, publication_year);
CREATE INDEX idx_authorships_author_work ON authorships (author_id, work_id);
CREATE INDEX idx_affiliations_institution ON affiliations (institution_id,
work_id, author_id);
CREATE INDEX idx_work_topics_topic_work ON work_topics (topic_id, work_id);
CREATE INDEX idx_work_topics_primary ON work_topics (topic_id, work_id)
WHERE is_primary_topic;
CREATE INDEX idx_work_sources_source_work ON work_sources (source_id, work_id);
CREATE INDEX idx_citations_cited_citing ON citations (cited_work_id,
citing_work_id);
CREATE INDEX idx_citations_citing_cited ON citations (citing_work_id,
cited_work_id);
```



# Neo4j Graph Architecture

## Index-Free Adjacency

- 🕒 **First-Class Graph Entities:** Works, Authors, Institutions, and Topics represented as labeled nodes.
- 🔗 **Typed Edge Relationships:** Direct pointer links for :AUTHORED, :AFFILIATED\_WITH, and :CITES.
- 🌐 **O(1) Pointer Chasing:** Traversing citations does not require expensive B-tree index lookups on junction tables.



# Dimensional Data Warehouse

## Star Schema Architecture (DFM)

Centered around fact\_work at the grain of **one single work**.

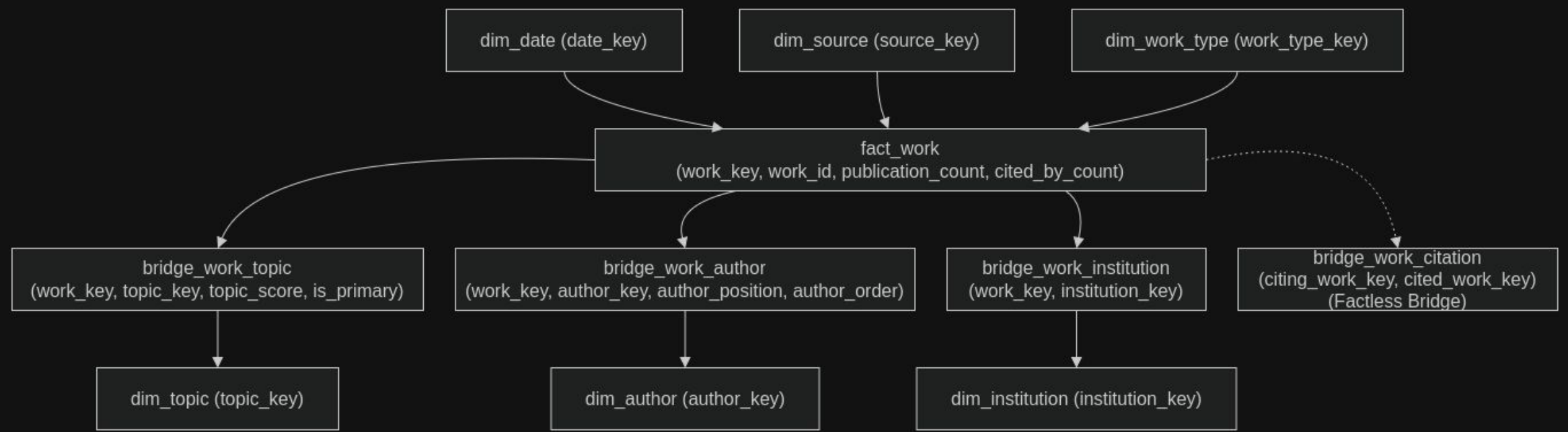
 **Conformed Dimensions:** dim\_date, dim\_source, dim\_work\_type.

 **Pre-Aggregated Measures:** Additive publication counts and semi-additive citation metrics.

## Multivalued Bridge Tables

Handles M:N relationships (e.g. multi-author or multi-topic works) without inflating fact row grain.

 **Double-Counting Prevention:** Fact table handles corpus-level totals; bridge tables are joined only during dimensional slicing.



# Closed-Network Integrity

## Pruning External References

Scholarly works cite thousands of external papers outside any finite sample boundary. Raw API responses initially yielded **8,850 dangling references**.

Allowing these into PostgreSQL causes Foreign Key violations, while in Neo4j it generates thousands of empty leaf nodes that corrupt path traversal algorithms.

- ✓ Zero foreign-key violations in PostgreSQL.
- ✓ Guaranteed reachability across multi-hop graph traversals.
- ✓ Prunes work self-citations while keeping author self-citations.

**ETL Boundary Rule:** Citations are strictly filtered to internal edges where both citing and cited works exist in the retained corpus.

# Contextual Affiliations

## The Conceptual Trap

Modeling affiliation as a direct timeless relationship:  
(:Author)-[:AFFILIATED\_WITH]->(:Institution)  
creates severe data distortion.

An author affiliated with Stanford in 2020 and OpenAI in 2023 would falsely attribute all 2020 papers to OpenAI.

## The Event-Based Solution

Affiliations are modeled strictly as **work-contextual events** using a ternary junction table:

**Relational:**

```
CREATE TABLE affiliations ( work_id TEXT, author_id TEXT,
institution_id TEXT, PRIMARY KEY (work_id, author_id,
institution_id), FOREIGN KEY (work_id, author_id) REFERENCES
authorships(work_id, author_id) );
```

**Graph:**

```
Relationship property (:Author)-[:AFFILIATED_WITH {work_id:
'W123'}]->(:Institution)
```

Preserves exact paper provenance across both relational and graph models.

# Evaluation query

| ID | Question                                                                                                                            |
|----|-------------------------------------------------------------------------------------------------------------------------------------|
| Q1 | How did the number of LLM/Generative-AI works and their median/average citations evolve by year, work type, and source?             |
| Q2 | Which authors, institutions, and sources are most productive or most cited, and how do rankings change by year and topic?           |
| Q3 | Which topic subfields and sources have the highest growth in publications and citation impact from 2020 to 2025?                    |
| Q4 | Which author-institution collaborations produce the most works and the highest citation impact, by year?                            |
| Q5 | Which LLM works are connected by citation paths of length 2–4, and what are the shortest paths between selected highly cited works? |
| Q6 | Which authors act as bridges between otherwise weakly connected co-authorship communities, and which institutions do they connect?  |

# Querying Variable Paths (Q5)

## SQL Recursive CTE (PostgreSQL)

```
WITH RECURSIVE citation_paths AS (  
  SELECT  
    c.citing_work_id AS start_work_id,  
    c.cited_work_id AS current_work_id,  
    ARRAY[c.citing_work_id, c.cited_work_id]::TEXT[] AS  
path,  
    1 AS hops  
  FROM citations c  
  UNION ALL  
  SELECT  
    p.start_work_id,  
    c.cited_work_id,  
    p.path || c.cited_work_id,  
    p.hops + 1  
  FROM citation_paths p  
  JOIN citations c ON c.citing_work_id = p.current_work_id  
  WHERE p.hops < 4  
    AND NOT c.cited_work_id = ANY (p.path)  
)  
SELECT hops, COUNT(*) AS path_count  
FROM citation_paths  
WHERE hops BETWEEN 2 AND 4  
GROUP BY hops  
ORDER BY hops;
```

Requires explicit array tracking and cycle guards.

## Cypher Pattern Matching (Neo4j)

```
MATCH p = (start:Work)-[:CITES*2..4]→(target:Work)  
WHERE reduce(seen = [], n IN nodes(p) |  
  CASE WHEN n IN seen THEN seen ELSE seen + n END  
  ) = nodes(p)  
RETURN length(p) AS hops, count(p) AS path_count  
ORDER BY hops;
```

Native variable-length path syntax with automatic cycle elimination.

# Aggregation & Slicing

## Collaboration Joins (Q4)

Finding inter-institutional co-authorship pairs:

```
MATCH (a1:Author)-[af1:AFFILIATED_WITH]→(i1:Institution)
MATCH (a1)-[:AUTHORED]→(w:Work {id: af1.work_id,
publication_year: 2020})
MATCH (a2:Author)-[af2:AFFILIATED_WITH]→(i2:Institution)
MATCH (a2)-[:AUTHORED]→(w)
WHERE i1.id < i2.id
WITH i1, i2, collect(DISTINCT w) AS works
RETURN i1.display_name AS institution_1,
       i2.display_name AS institution_2,
       size(works) AS collaborative_work_count,
       reduce(total = 0, w IN works | total +
w.cited_by_count) AS summed_work_citations
ORDER BY collaborative_work_count DESC, summed_work_citations
DESC
LIMIT 30;
```

## Dimensional Slicing (Q2B)

Star Schema institutional productivity roll-up:

```
SELECT
    i.institution_id,
    i.display_name,
    SUM(f.publication_count) AS work_count,
    SUM(f.cited_by_count) AS citations
FROM fact_work f
JOIN bridge_work_institution bwi ON bwi.work_key = f.work_key
JOIN dim_institution i ON i.institution_key =
bwi.institution_key
GROUP BY i.institution_id, i.display_name
ORDER BY work_count DESC, citations DESC, i.institution_id
LIMIT 20;
```

# Co Authorship (Q6)

## SQL Recursive (PostgreSQL)

```
WITH coauthor_edges AS (  
    SELECT DISTINCT  
        LEAST(left_side.author_key, right_side.author_key) AS  
author_1_key,  
        GREATEST(left_side.author_key, right_side.author_key)  
AS author_2_key  
    FROM bridge_work_author left_side  
    JOIN bridge_work_author right_side  
        ON right_side.work_key = left_side.work_key  
        AND right_side.author_key > left_side.author_key  
) , author_degree AS (  
    SELECT author_1_key AS author_key, COUNT(*) AS degree  
FROM coauthor_edges GROUP BY author_1_key  
    UNION ALL  
    SELECT author_2_key AS author_key, COUNT(*) AS degree  
FROM coauthor_edges GROUP BY author_2_key  
)  
SELECT a.author_id, a.display_name, SUM(d.degree) AS  
coauthor_degree  
FROM author_degree d  
JOIN dim_author a ON a.author_key = d.author_key  
GROUP BY a.author_id, a.display_name  
ORDER BY coauthor_degree DESC, a.author_id  
LIMIT 20;
```

## Cypher Pattern Matching (Neo4j)

```
MATCH  
(a1:Author)-[:AUTHORED]→(w:Work)←[:AUTHORED]-(a2:Author)  
WHERE a1 < a2  
WITH a1, collect(DISTINCT a2) AS coauthors  
RETURN a1.id AS author_id,  
        a1.display_name AS author_name,  
        size(coauthors) AS coauthor_degree  
ORDER BY coauthor_degree DESC, author_id  
LIMIT 20;
```

# Empirical Benchmark Results

| Workload / Analytical Task            | PostgreSQL 3NF (ms) | Star Schema DW (ms) | Neo4j LPG (ms) | Winning Paradigm |
|---------------------------------------|---------------------|---------------------|----------------|------------------|
| Q1: Yearly LLM Work Evolution         | 2.18 ms             | 1.96 ms             | 12.21 ms       | WAREHOUSE        |
| Q2A: Top 20 Author Productivity       | 60.37 ms            | 18.81 ms            | 29.60 ms       | WAREHOUSE        |
| Q2B: Top 20 Institutional Impact      | 11.83 ms            | 5.90 ms             | 56.32 ms       | WAREHOUSE        |
| Q3: Topic-Field Roll-Up               | 4.97 ms             | 2.44 ms             | 7.29 ms        | WAREHOUSE        |
| Q4: Institutional Collaboration Pairs | 10.93 ms            | 4.22 ms             | 22.59 ms       | WAREHOUSE        |
| Q5: Directed Citation Paths 2-4 Hops  | 1585.86ms           | 1272.42ms           | 1413.41ms      | WAREHOUSE        |
| Q6: Co-Authorship Bipartite Expansion | 381.86 ms           | 235.48 ms           | 124.23 ms      | GRAPH (NEO4J)    |

# A strange result

We designed Q5 to be suitable for graphDB but turned out warehouse won

| Workload / Analytical Task           | PostgreSQL 3NF (ms) | Star Schema DW (ms) | Neo4j LPG (ms) | Winning Paradigm |
|--------------------------------------|---------------------|---------------------|----------------|------------------|
| Q5: Directed Citation Paths 2-4 Hops | 1585.86ms           | 1272.42ms           | 1413.41ms      | WAREHOUSE        |

## Path explosion

The tentative of the graph database to explore all possible paths causes it to try to explore millions of path while warehouse produces join on the edge table citation\_path, a relatively small table

## No recursive check

the no recursive path check actually increase the workload of the query

```
WHERE reduce(seen = [], n IN nodes(p) |
            CASE WHEN n IN seen THEN seen
            ELSE seen + n END
            ) = nodes(p)
```

# LLM Research Insights



## Publication evolution

Preprints (435 works) outpaced peer-reviewed articles (203 works) by >2:1 in 2020. While in 2025 article sits to 733 while review to 58.

We can also notice the peak of citations for article in 2023



## Top Impact Authors

|                                                                                                    |           |
|----------------------------------------------------------------------------------------------------|-----------|
|  Jianfeng Gao     | 27   4799 |
|  Luke Zettlemoyer | 25   1968 |
|  Yejin Choi       | 24   1548 |



## Citation Cascades

Multi-hop graph paths revealed strong 2-hop (100607) and 3-hop (375290) clustering radiating outward from papers

# Questions & Discussion

OpenAlex LLM Analyzer — Multi-Paradigm Comparison Complete

Pair Programming & Comparative Database Systems Study • 2026